Linux Pizza Palace

A delicious analogy for understanding the Linux CPU scheduler

🍕 Pizza Order

= Process / Thread

👨‍🍳 Chef

= CPU Core

📋 Manager

= Linux Scheduler

The Setup

The Kitchen

2 chefs — 2 CPU cores

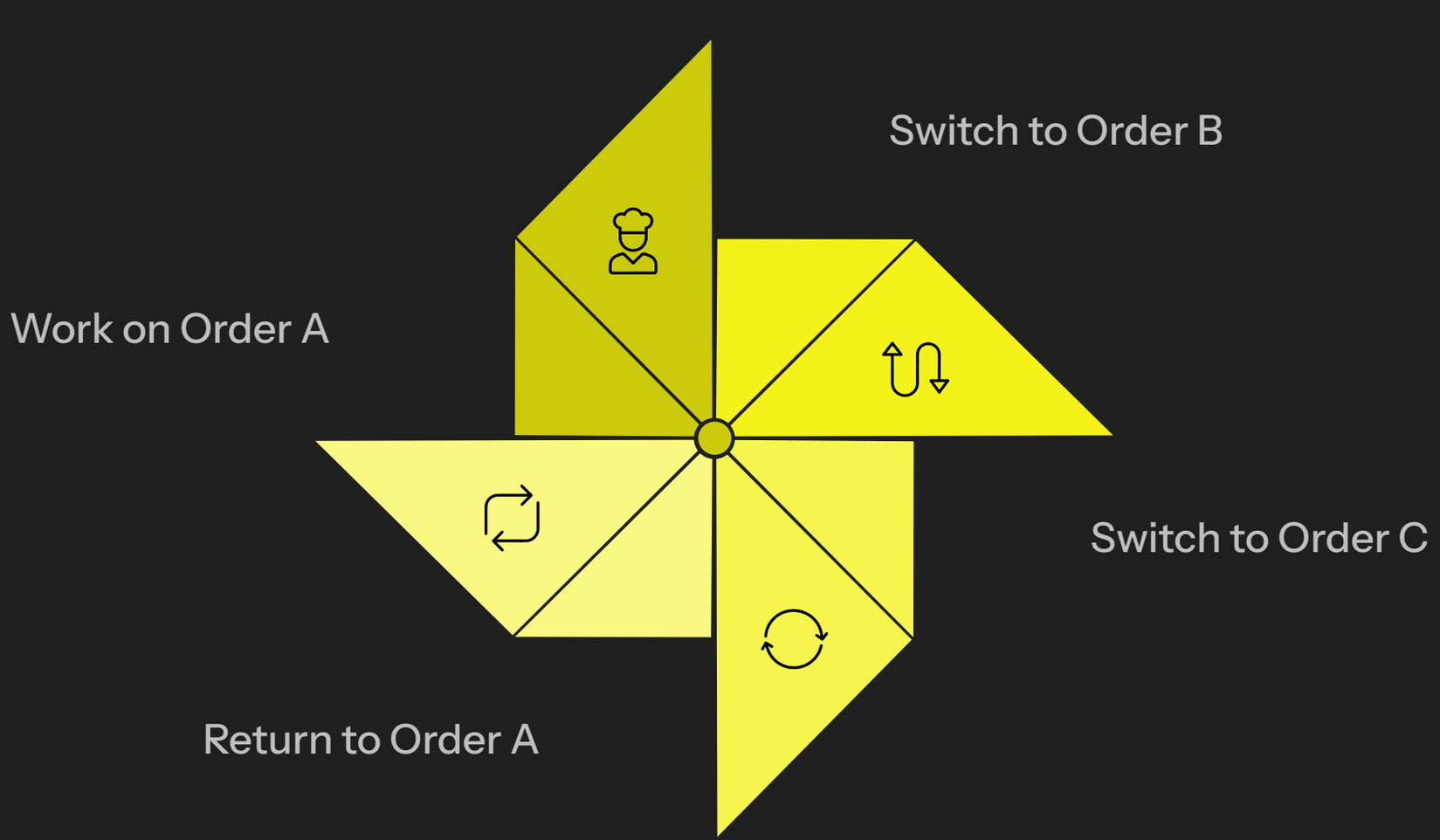
20 orders waiting in queue

Order Types

- Simple cheese pizzas
- Giant catering orders
- ⚡ VIP rush orders

Who gets chef time next, for how long, and on which chef?

Time Slices = Chef Attention



That short window of attention is a **time slice** (time quantum). No single massive order blocks the whole kitchen — or the whole CPU.

Context Switching

When a chef stops one pizza and starts another:

- → Wipes hands
- → Checks new recipe
- → Grabs new ingredients

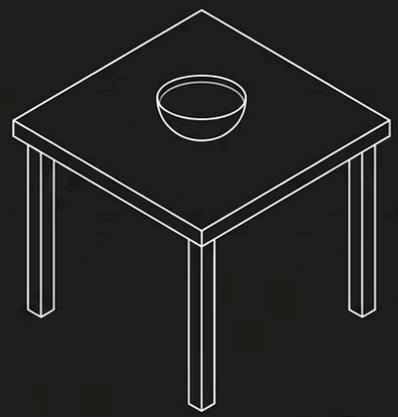
⚠ That overhead is a **context switch**. Linux minimises unnecessary switching — it takes real time.

Fair Scheduling (CFS)

The **Completely Fair Scheduler** tracks how much chef time each order has already received.

ORDER A
(LOW PRIORITY)

Chef Time: 10 mins



B

ORDER B
(HIGH PRIORITY NEXT)

Chef Time: 2 mins

Virtual Runtime

Linux keeps a score for every process.

Lower score = less CPU time received

Lower score = scheduled sooner

Fairness is baked in at the kernel level.

Priority & Nice Values

nice -20

URGENT VIP ORDER

Jumps to the front

nice 0

Regular customer

Standard queue

nice +19

Work on this... eventually

Lowest priority

Real-Time Scheduling

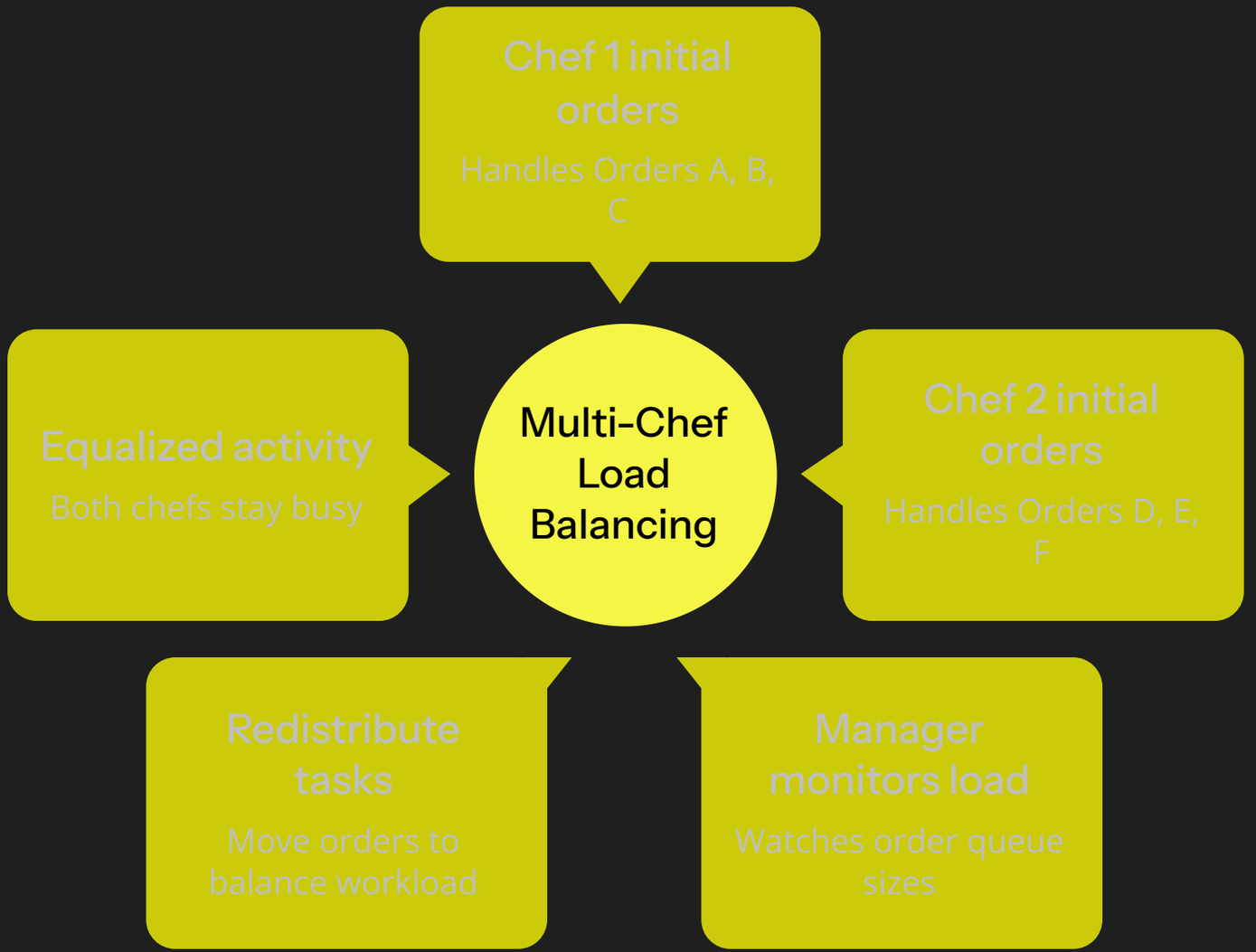
Some orders have a hard deadline. Late = disaster.

> Linux **real-time schedulers** let critical jobs jump the queue entirely — fairness is irrelevant when timing is everything.

Real-world examples:

- Audio processing
- Robotics control
- Industrial systems

Multi-Core = Multiple Chefs



Load balancing ensures no chef idles while another drowns in orders. The manager constantly rebalances — just as Linux migrates processes across CPU cores.

CPU Cache Affinity

A chef remembers their half-finished pizza — the sauces, the toppings, the position of every tool.

Moving that order to a **new chef** means starting from scratch. Slower, costly, wasteful.

Linux prefers keeping a process on the **same CPU core** to preserve warm cache data. This is called **CPU affinity**.

I/O Wait

🍕 In the Kitchen

Pizza needs dough from the storeroom. Chef doesn't stand idle — they start another order while waiting.

💻 In Linux

Process waiting on disk, network, or input is paused. CPU immediately picks up another runnable process.

CPUs stay busy. Waiting is never wasted.

Starvation

If VIP orders never stop arriving, regular customers **never get served**.

> ⚠ That's **starvation** — a process that never receives CPU time because higher-priority work always wins.

How Linux fights it:

- CFS virtual runtime naturally ages waiting processes
- Everyone eventually gets a turn

Putting It All Together

Linux scheduling is one manager, balancing it all in real time:

Fairness

CFS virtual runtime

Responsiveness

Real-time schedulers

Priority

Nice values

Efficiency

Cache affinity & load balancing

Low Overhead

Minimal context switches

> The Linux scheduler is just a very fast, very determined pizza manager. 🍕